LINUXBLOG.IO · ESSENTIAL REFERENCE

100+ Linux Commands

The essential reference for sysadmins, developers, and power users — with examples and deep-dive links.

SYSADMIN DEVOPS CLI REFERENCE

```
hayden@linuxblog:~$ cat ./100-commands.pdf
# A curated guide to the commands every Linux user
# should know — from file ops to performance monitoring.
hayden@linuxblog:~$
```

Hayden James · LinuxBlog.io

00 / INTRO

Welcome

This is a curated reference to **100+ essential Linux commands** used daily by sysadmins, developers, and power users. It focuses on native Unix tools and widely-available CLI utilities — the ones that earn their place on every server and workstation.

How to use this guide. Commands are grouped by category so you can scan by intent ("I need to check disk usage" → *File Management & Disk Usage*). Each entry has a one-line purpose, a short real-world example, and a link when a deep-dive tutorial exists on LinuxBlog.io.

Skill level. Mixed. The basics (`ls`, `cd`, `cp`) are here for completeness; the meat is in less-obvious tools like `ss`, `journalctl`, `sar`, and `ncdu` that separate comfortable CLI users from people who still open a GUI file manager when they forget where `du` lives.

Examples are minimal on purpose. Most are a single line showing the most-used flag combo, not a full tutorial. When you hit something you want to explore deeper, follow the link.

→ JOIN THOUSANDS OF LINUX ENTHUSIASTS

Questions on any command? Share your own scripts and configs? **LinuxCommunity.io** — our free forum for Linux users. Registered members get access to members-only PDFs, configs, and exclusive guides. Signup is 30 seconds.

SHARING THIS PDF

Free for LinuxCommunity.io members — and please help keep it that way. Instead of forwarding the file, **share the signup link** so friends can get the latest version (we update occasionally) and join the community. © Hayden James · LinuxBlog.io. Personal use only; not for redistribution or commercial reuse.

TOC

Contents

111 commands, grouped into 8 working categories. Scan the bookmarks in your PDF reader to jump.

01	System Monitoring & Performance	15 cmds
02	Networking & Connectivity	17 cmds
03	File Management & Disk Usage	18 cmds
04	System & Process Management	16 cmds
05	User Management & Security	10 cmds
06	Text, Scripts & Shell Tools	18 cmds
07	Disks, Partitions & Devices	11 cmds
08	Bonus: CLI Power Tools	6 cmds
09	Keep Reading	6 picks
↗	Join LinuxCommunity.io	free

01

System Monitoring & Performance

When the server feels slow, these are the tools you reach for first. Start with `top` or `htop` for a live view, then drill into whichever subsystem is struggling — CPU, memory, disk I/O, or network.

$ top LIVE

System-wide process and resource view. Still on every Linux box.

```
top -o %CPU   # sort by CPU usage
```

↗ Deep-dive: top command

$ htop LIVE

Interactive process viewer — colors, mouse support, tree view. What you install five minutes after setting up a new box.

```
htop -t   # tree view of processes
```

↗ Deep-dive: htop customization

$ atop HISTORICAL

Advanced performance monitor — logs all activity to disk, so you can scroll back to "what was this server doing at 3am last Tuesday?"

```
atop -r /var/log/atop/atop_$(date +%Y%m%d)
```

↗ Deep-dive: atop for performance analysis

$ vmstat SNAPSHOT

Memory, process, paging, block I/O, and CPU stats. Sample every N seconds.

```
vmstat 2 5   # 5 samples, 2s apart
```

↗ Deep-dive: vmstat + friends

$ dstat ALL-IN-ONE

Combines `vmstat`, `iostat`, `netstat`, `ifstat` into one tool. Color-coded live stream.

```
dstat -tcmnd --top-cpu
```

↗ Deep-dive: dstat examples

$ iotop DISK I/O

Which process is hammering your disk. Requires root.

```
sudo iotop -oPa   # only active, cumulative
```

↗ Deep-dive: iotop examples

$ iostat DISK I/O

CPU and disk I/O stats. Useful for spotting saturated disks.

```
iostat -xz 2   # extended, skip idle
```

↗ Deep-dive: iostat examples

$ free MEMORY

RAM and swap summary. The `-h` flag gives you human-readable units.

```
free -h
```

↗ Deep-dive: measuring memory usage

$ uptime LOAD

How long the system has been up + load averages at 1, 5, and 15 minutes.

```
uptime
```

↗ Deep-dive: uptime command

$ sar HISTORICAL

Collects and reports system activity (CPU, memory, disk, network) — over time, not just live. Great for post-mortem.

```
sar -u 1 5   # CPU, 5 samples 1s apart
```

↗ Deep-dive: sar + RAM checks

$ ps PROCESSES

Snapshot of current processes. The aux combo is near-universal muscle memory.

```
ps aux --sort=-%mem | head
```

$ pstree PROCESSES

Process tree — shows parent/child relationships. Tells you exactly what spawned that mystery process.

```
pstree -p
```

↗ Deep-dive: pstree examples

$ w SESSIONS

Who is logged in, what they're doing, and the system load. One-letter command, surprisingly packed.

```
w
```

$ last AUDIT

Login history from `/var/log/wtmp`. Who logged in, when, from where, for how long.

```
last -n 10
```

↗ Deep-dive: last command

$ glances / $ nmon ALTERNATIVES

Drop-in alternatives to `top`/`htop` with richer dashboards. `glances` can export to InfluxDB/Prometheus; `nmon` has a classic sysadmin feel.

```
glances   #   nmon
```

02

Networking & Connectivity

Interfaces, routes, sockets, DNS, and HTTP(S) tools. The old guard (`ifconfig`, `netstat`, `route`) still work on most systems but `ip` and `ss` are the modern replacements you should be using.

$ ip INTERFACES

Modern replacement for `ifconfig` and `route`. Controls interfaces, addresses, routes, and more.

```
ip -br addr   # brief interface summary
```

↗ Deep-dive: ip from iproute2

$ ss SOCKETS

Socket statistics — the `netstat` replacement. Faster, more detailed.

```
ss -tulpn   # all listening TCP/UDP sockets + pids
```

↗ Deep-dive: ss examples

$ netstat LEGACY

Still works, still in many scripts. Being slowly replaced by `ss`.

```
netstat -rn   # routing table
```

↗ Deep-dive: netstat examples

$ ping REACHABILITY

ICMP echo to test reachability and latency. The default everywhere.

```
ping -c 4 8.8.8.8
```

↗ Deep-dive: ping examples

$ traceroute / $ mtr PATH

Trace the path packets take. `mtr` is live and continuously updates, which is usually what you actually want.

```
mtr -rw google.com
```

↗ Deep-dive: traceroute examples

$ iftop BANDWIDTH

Real-time per-connection bandwidth monitor. Which flow is using your pipe right now.

```
sudo iftop -i eth0
```

↗ Deep-dive: iftop examples

$ nethogs BANDWIDTH

Like `iftop` but groups bandwidth by *process* — so you know which binary is saturating your upload.

```
sudo nethogs
```

$ nc SWISS ARMY

Netcat — raw TCP/UDP client and server. Test if a port is open, move files quickly, or debug a protocol.

```
nc -zv host 22   # port scan
```

$ nmcli NETWORKMANAGER

CLI control for NetworkManager on desktops and many servers. Connect to WiFi, manage VPNs, edit connections.

```
nmcli device wifi list
```

$ dig / $ host / $ nslookup DNS

DNS lookup tools. `dig` is the sysadmin's favorite (detailed output), `host` is terse, `nslookup` is cross-platform muscle memory.

```
dig +short A linuxblog.io
```

$ whois REGISTRATION

Domain and IP ownership lookup. Good for figuring out who owns the IP hammering your firewall.

```
whois 1.1.1.1
```

$ wget DOWNLOAD

Retrieve files over HTTP(S) and FTP. Resume-friendly, recursive-friendly.

```
wget -c https://example.com/file.iso
```

$ curl REQUESTS

Transfer data over many protocols. The default HTTP-debugging tool and a scripting workhorse.

```
curl -sI https://example.com   # head only
```

↗ Deep-dive: curl command

$ ssh REMOTE

Secure remote shell. The command you use more than any other once you run real servers.

```
ssh -J bastion user@private-host
```

↗ Deep-dive: SSH security

$ scp TRANSFER

Copy files over SSH. Simple one-shot transfers; for anything ongoing use `rsync`.

```
scp file.tgz user@host:/tmp/
```

↗ Deep-dive: scp examples

$ rsync SYNC

Differential sync. Only transfers changes. Essential for backups and moving large trees.

```
rsync -aHAX --delete src/ dest/
```

↗ Deep-dive: rsync examples

$ tcpdump CAPTURE

Packet capture on the CLI. Drop into it when things are weird at the protocol level.

```
sudo tcpdump -ni eth0 port 443
```

03

File Management & Disk Usage

Creating, moving, finding, and archiving. The daily bread of any CLI session. Most of these you already know — they're here for completeness and a few flag tricks you may not have seen.

$ ls LIST

List directory contents. The `-lahtr` combo is a surprisingly good default.

```
ls -lahtr   # long, human, hidden, time-sorted, reverse
```

↗ Deep-dive: ls examples

$ cd / $ pwd NAVIGATE

Change directory and print the current one. `cd -` jumps back to your previous location.

```
cd -   # toggle with prev dir
```

↗ Deep-dive: navigating with cd

$ cp COPY

Copy files or directories. `-a` preserves everything (permissions, symlinks, timestamps).

```
cp -a src/ dest/
```

↗ Deep-dive: cp command

$ mv MOVE/RENAME

Move or rename. Also handles "rename many with a pattern" via brace expansion.

```
mv logs.{txt,txt.bak}
```

↗ Deep-dive: mv examples

$ rm DELETE

Remove files. There is no trash can. Measure twice, `rm -rf` once.

```
rm -i *.tmp   # prompt before each
```

↗ Deep-dive: rm examples

$ mkdir CREATE

Make directories. -p creates parents as needed and doesn't complain if the target exists.

```
mkdir -p a/b/c
```

↗ Deep-dive: mkdir examples

$ touch STAMP/CREATE

Update file access/modification times — or create an empty file if it doesn't exist.

```
touch placeholder.txt
```

↗ Deep-dive: touch examples

$ df DISK

Disk space usage per mountpoint. -h is human-readable, -T shows filesystem type.

```
df -hT
```

↗ Deep-dive: df examples

$ du USAGE

Disk usage of files/directories. For finding what's eating space.

```
du -sh * | sort -h   # sorted sizes
```

↗ Deep-dive: du examples

$ ncdu DISK

Interactive du — navigable, sortable, deletable. Often faster than trying to eyeball du -h output.

```
ncdu /var
```

↗ Deep-dive: ncdu for large dirs

$ find SEARCH

Find files by name, size, date, permissions, owner — with actions. The most flexible tool on this list.

```
find . -name '*.log' -mtime +30 -delete
```

↗ Deep-dive: find examples

$ locate SEARCH

Instant file-name search using a prebuilt index (`updatedb`). Faster than `find` but only as fresh as the last index.

```
locate sshd_config
```

$ tar ARCHIVE

Create and extract archives. Combine with `gzip`/`bzip2`/`xz` for compression.

```
tar -czvf backup.tar.gz /etc/
```

↗ Deep-dive: tar examples

$ gzip / $ bzip2 / $ xz COMPRESS

File compression. `gzip` is fastest, `xz` compresses hardest, `bzip2` is the old middle ground.

```
gzip -9 large.log   # max compression
```

↗ Deep-dive: gzip a directory

$ zip / $ unzip ARCHIVE

When you need to produce something Windows/Mac can open without extra tools.

```
zip -r archive.zip folder/
```

$ ln LINKS

Create hard or symbolic links. `-s` for symlinks (the common case).

```
ln -s /opt/app/current /var/app
```

$ file INSPECT

Identify a file's type by content, not extension. Useful for "what even is this binary."

```
file mystery.bin
```

$ stat INSPECT

Detailed info about a file: size, permissions, access/modify/change times, inode, and more.

```
stat /etc/hosts
```

04

System & Process Management

Starting, stopping, scheduling, and sticking. The tools to manage what's running on your box and the services that keep it running.

$ systemctl SERVICES

The control surface for systemd. Start, stop, enable, mask services — and query their state.

```
systemctl status nginx
```

$ journalctl LOGS

Query the systemd journal. Far more flexible than tailing `/var/log/*` directly.

```
journalctl -u nginx -f   # follow a unit
```

↗ Deep-dive: journalctl guide

$ kill SIGNAL

Send a signal to a process by PID. Default is SIGTERM. SIGKILL (-9) is the last resort.

```
kill -HUP 1234   # reload config
```

$ killall / $ pkill SIGNAL

Signal by name (or pattern, with `pkill`). Handy when you don't want to look up a PID.

```
pkill -f 'python bad_script.py'
```

$ nice / $ renice PRIORITY

Adjust CPU scheduling priority of a process. Lower nice = higher priority.

```
nice -n 19 ./slow-job.sh
```

$ nohup DETACH

Run a command immune to HUP signals — survives your SSH session ending.

```
nohup ./long.sh > out.log 2>&1 &
```

$ screen SESSION

Keep a shell session running on a remote server even if you disconnect. Older than `tmux` but still everywhere.

```
screen -S work   # then Ctrl+A D to detach
```

$ tmux SESSION

Modern terminal multiplexer. Panes, windows, sessions, scripting. Pick one of `screen`/`tmux` and get fluent.

```
tmux new -s dev
```

↗ Deep-dive: tmux guide

$ cron / $ crontab SCHEDULE

Schedule commands to run on a cadence. Still the default scheduler on most systems.

```
crontab -e   # edit your user's crontab
```

$ at SCHEDULE

Run a command once at a specific time. The one-shot cousin of `cron`.

```
echo './task.sh' | at now + 2 hours
```

$ sleep PAUSE

Pause for N seconds (or m/h/d). The glue between script steps.

```
sleep 5m
```

$ wait BACKGROUND

Wait for background jobs to finish before continuing. Essential for parallel scripts.

```
./a.sh & ./b.sh & wait
```

$ dmesg KERNEL

Kernel ring buffer — driver messages, hardware events, OOM kills. On modern systems try `-wH`.

```
sudo dmesg -wH
```

$ lsof INSPECT

List open files (and sockets, pipes, devices) per process. Indispensable for "what's holding this file?"

```
lsof -i :80   # who has port 80?
```

$ strace DEBUG

Trace system calls and signals a process makes. When a program misbehaves and logs don't help, this shows exactly what it's asking the kernel for.

```
strace -f -e trace=openat ./app
```

↗ Deep-dive: strace for debugging

$ watch MONITOR

Run a command repeatedly and show the output full-screen. The poor-man's live dashboard.

```
watch -n 1 'free -h; ss -s'
```

05

User Management & Security

Accounts, passwords, permissions, ownership. The baseline tools you need to understand before getting fancy with PAM, sudoers, or SELinux.

$ sudo ELEVATE

Execute a command as another user (usually root). The modern replacement for `su -c`.

```
sudo -i   # interactive root shell
```

↗ Deep-dive: sudo examples

$ passwd PASSWORD

Change a user's password — your own, or (as root) anyone's.

```
sudo passwd alice
```

$ useradd ACCOUNT

Create a new user account. Use `-m` to create a home directory, `-s /bin/bash` for a shell.

```
sudo useradd -m -s /bin/bash alice
```

$ usermod ACCOUNT

Modify an existing user — shell, groups, home directory, expiration.

```
sudo usermod -aG sudo alice
```

$ userdel ACCOUNT

Delete a user. `-r` also removes their home dir and mail spool.

```
sudo userdel -r alice
```

$ chmod PERMISSIONS

Change file permission bits. Numeric (755) or symbolic (u+x) — whichever sticks in your head.

```
chmod +x deploy.sh
```

↗ Deep-dive: chmod, chown, umask

$ chown PERMISSIONS

Change ownership of files/directories. `user:group` syntax; `-R` for recursive.

```
sudo chown -R www-data:www-data /var/www
```

↗ Deep-dive: chmod, chown, umask

$ umask PERMISSIONS

Default permission mask for newly-created files. `022` = new files world-readable, `077` = only you.

```
umask 077
```

↗ Deep-dive: chmod, chown, umask

$ chroot ISOLATE

Run a process with a different apparent root directory. Core building block of containers (before we had proper namespaces).

```
sudo chroot /mnt/rescue /bin/bash
```

$ id IDENTITY

Print the user and group IDs for yourself or another user. Useful in scripts and debugging "why can't I read this?"

```
id www-data
```

06

Text, Scripts & Shell Tools

Viewing, editing, searching, transforming. The trio `grep` + `awk` + `sed` deserves deep study on its own — it's the swiss-army for turning structured text into anything else.

$ cat VIEW

Concatenate and print files. Often misused for short files where `less` would be better.

```
cat /etc/os-release
```

$ less VIEW

Page through files. Scroll with arrows, search with `/`, follow with Shift+F (like `tail -f`).

```
less +F /var/log/syslog
```

$ more / $ tac VIEW

`more` is `less`'s older, simpler sibling. `tac` is `cat` reversed — last line first.

```
tac access.log | head   # newest first
```

$ tail VIEW

Show the last N lines of a file. With `-f`, follow new writes (classic log-watching).

```
tail -fn 100 /var/log/nginx/error.log
```

$ head VIEW

First N lines. Default 10. Pair with `|` to preview large outputs.

```
ps aux | head -20
```

$ grep SEARCH

Search text for a pattern. `-r` recurses, `-i` ignores case, `-n` shows line numbers, `-E` enables extended regex.

```
grep -rIn 'TODO' src/
```

↗ Deep-dive: grep examples

$ awk TRANSFORM

Pattern-scanning language. Extract columns, sum them, reformat lines. Once you see it, you use it weekly.

```
awk '{sum+=$3} END{print sum}' access.log
```

↗ Deep-dive: awk practical guide

$ sed TRANSFORM

Stream editor. Substitute, delete, insert lines. The "quick find-and-replace on a file" tool.

```
sed -i 's/old/new/g' config.yml
```

↗ Deep-dive: sed examples

$ cut / $ sort / $ uniq PIPE GLUE

The unsung heroes. Cut columns, sort lines, de-dup adjacent repeats (`sort | uniq -c | sort -rn` is a near-universal top-N idiom).

```
cut -d: -f1 /etc/passwd | sort | uniq
```

$ xargs PIPE GLUE

Build and execute command lines from stdin. The bridge that turns "a list of things" into "commands run on each thing." Pairs beautifully with `find` and `grep -l`.

```
find . -name '*.log' | xargs -r rm
```

↗ Deep-dive: xargs examples

$ vi / $ vim EDIT

The modal editor that's on every Unix system. Learn enough to survive: `i`, `Esc`, `:wq`, `:q!`, `/pattern`.

```
vim +/ERROR app.log
```

$ nano EDIT

The modeless alternative. Commands visible at the bottom. Fine for config tweaks when you don't want to vim.

```
sudo nano /etc/nginx/nginx.conf
```

$ man / $ apropos HELP

The manual pages — authoritative but dense. `apropos` searches man page descriptions when you know the task but not the command.

```
apropos 'compress'
```

↗ Deep-dive: man examples

$ tldr HELP

Community-maintained examples for common commands. Answers "just show me how people actually use this."

```
tldr tar
```

$ history SHELL

Your command history. `!NNN` reruns command N; `Ctrl+R` interactive search is often better.

```
history | grep docker
```

$ alias SHELL

Create shortcuts for commands. Put the ones you like in `~/.bashrc` or `~/.zshrc`.

```
alias ll='ls -lahtr'
```

↗ Deep-dive: bash aliases

$ env SHELL

Show or modify environment variables for a command. `env | sort` is the "what's in my environment" command.

```
env FOO=bar ./app
```

$ clear SHELL

Clear the terminal. `Ctrl+L` does the same thing faster.

```
clear
```

07

Disks, Partitions & Devices

Mounting, partitioning, formatting, inspecting. Most of these are tools you don't touch daily but absolutely need when setting up a new disk, rescuing a broken system, or troubleshooting hardware.

$ mount / $ umount FILESYSTEMS

Attach (and detach) a filesystem at a mountpoint. Without args, `mount` lists everything currently mounted.

```
sudo mount /dev/sdb1 /mnt/data
```

$ lsblk INSPECT

Tree view of block devices. Fastest way to see "what disks and partitions do I have?"

```
lsblk -f   # include filesystem info
```

$ blkid INSPECT

Show block device attributes — UUIDs, labels, filesystem types. What you grep for when writing `/etc/fstab`.

```
sudo blkid
```

$ fdisk PARTITION

Classic MBR/GPT partition editor. Interactive; `-l` just lists.

```
sudo fdisk -l
```

$ parted PARTITION

Like `fdisk` but scriptable and GPT-first. Better for automation.

```
sudo parted /dev/sdb print
```

$ mkfs FORMAT

Create a filesystem on a partition. Common variants: `mkfs.ext4`, `mkfs.xfs`, `mkfs.btrfs`.

```
sudo mkfs.ext4 -L data /dev/sdb1
```

$ fsck REPAIR

Check and repair a filesystem. Unmount first. Run from rescue media for system disks.

```
sudo fsck -f /dev/sdb1
```

$ lspci HARDWARE

List PCI devices. Useful when troubleshooting GPU, NIC, or RAID controller issues.

```
lspci | grep -i network
```

$ lsusb HARDWARE

List USB devices. Quick way to check if a drive/device is being detected at all.

```
lsusb
```

$ dd LOW-LEVEL

Raw block-level copy. Writes disk images, clones drives, benchmarks — and destroys data if you get `of=` wrong.

```
sudo dd if=disk.img of=/dev/sdX bs=4M status=progress
```

↗ Deep-dive: benchmarking with dd

$ smartctl HEALTH

Query drive SMART data. Catches failing disks before they fail.

```
sudo smartctl -a /dev/sda
```

08

Bonus: CLI Power Tools

Not always installed by default, but worth a quick `apt install` / `dnf install` on any box you'll live on. These turn a bare-bones server into a much more pleasant place to work.

$ btop INSTALL

The `htop` alternative on the cover of this PDF. Prettier graphs, mouse support, every stat you could want.

```
sudo apt install btop   #   btop
```

↗ Deep-dive: btop — the htop alternative

$ bashtop INSTALL

Predecessor to `btop`. Still installed on older boxes where `btop` isn't packaged yet.

$ nload INSTALL

Real-time per-interface bandwidth graphs. Tiny, focused, works over SSH.

```
sudo apt install nload
```

$ ncdu INSTALL

Covered earlier but worth flagging — not installed by default on minimal images. Install it on every server.

$ cheat INSTALL

Create and view your own cheatsheets on the CLI. Great place to keep "the way I actually use this command" notes.

```
cheat tar
```

$ fd INSTALL

A user-friendly `find`. Sensible defaults (respects .gitignore), color output, fast.

```
fd -e log   # all .log files, recursive
```

09 / NEXT

Keep Reading

Six articles from LinuxBlog.io that pair naturally with this reference. Start with the first — it's the closest companion to what you just read.

01

50 Essential Linux Commands You Should Know

The closest companion to this PDF — a narrative walkthrough of the 50 commands every Linux user should have in their muscle memory, with explanations of *why*, not just what.

02

Linux Networking Commands & Scripts

Goes deep on Chapter 02 of this PDF. Real-world networking recipes — diagnostics, bandwidth tests, DNS, firewalls, scripts you'll actually reuse.

03

Linux Sysadmin Tools

A curated tour of the tools sysadmins reach for beyond the built-in commands — observability, automation, backup, security. The layer above this PDF.

04

Linux Terminal Emulators

Where you run everything in this PDF. A comparison of modern terminals — Kitty, Alacritty, WezTerm, and the classics — with features that actually matter.

05

50 Linux Text Editors

Once you've outgrown `vi` and `nano`, there's a whole world. Terminal editors, GUI editors, modal beasts, minimalist ones — 50 options worth knowing.

06

Home Lab Beginner's Guide — Hardware

For when you want your own box to run these commands on. Hardware picks, form factors, quiet vs. powerful, new vs. used — the aspirational closer.

— KEEP GOING —

Join thousands of Linux enthusiasts.

LinuxCommunity.io is our free forum for sysadmins, developers, and Linux power users. Ask questions, share configs, trade war stories, and unlock more members-only downloads like this one.

linuxcommunity.io

// 01

Free forever

30-second signup, forum access immediately.

// 02

Members-only PDFs

More cheatsheets, configs, and guides unlock after signup.

// 03

Real sysadmins

Ask a question — get answers from people actually running Linux.

Share this PDF responsibly — please don't redistribute the file.
linuxcommunity.io/invites/2dPQd2ReUk is the friendly way.

© Hayden James · LinuxBlog.io · LinuxCommunity.io

Linux® is a registered trademark of Linus Torvalds. Not affiliated with the Linux Foundation.